



**ITM SKILLS
UNIVERSITY**

School of Future Tech

Case Study Report

on

Case Study 60: Telecom SIM Card Activation & Billing System

(Telecom BSS — JavaFX)

by

Yash Tambade

Roll No: 150096725173

Java Programming | B.Tech CSE 2025-29 | Semester III

ITM Skills University

Index

1. Introduction to the Case Study
2. Problem Statement / Case Background (Abstract)
3. Case Study Design
4. Methods, Algorithms & Technology Applied
5. Implementation Details and Snapshots
6. Results and Conclusion
7. References

1. Introduction to the Case Study

Telecom operators handle large volumes of customer, SIM card, plan, usage and payment data every day. Activating a SIM, tracking calls, SMS and data consumption, and producing accurate monthly bills are core Business Support System (BSS) functions that must be reliable, validated and easy to use.

This case study presents the design and implementation of **Telecom BSS — SIM Activation & Billing System**, a desktop application built with Java 21 and JavaFX 21. It allows an operator to register customers, activate SIM cards, manage prepaid and postpaid plans, record usage, generate itemised invoices, record payments and view billing reports.

The project demonstrates how core Java concepts — classes and objects, constructors, enums, the Collections Framework, StringBuilder, exception handling and input validation — combine to solve a realistic telecom billing workflow.

2. Problem Statement / Case Background (Abstract)

Background

Manual or spreadsheet-based record keeping in telecom operations leads to duplicate SIM registrations, wrong plan assignment, missing usage records and billing errors. A structured application that enforces validation rules and calculates charges from plan limits and recorded usage removes most of these problems.

Problem Statement

A telecom operator needs a system to manage SIM card activation, recharge plans, call/data usage tracking, and monthly billing for prepaid and postpaid customers. The system should generate bills based on plan type and usage.

Objectives

- Maintain customer and SIM card information.
- Manage recharge/subscription plans.
- Track call, SMS and data usage.
- Generate monthly bills.
- Search and sort customer records.
- Provide a telecom billing GUI.

Abstract

The Telecom BSS application is organised into model, repository, service, utility and presentation layers. Customers, SIMs, plans, usage records, bills and payments are held in Java collections: an ArrayList for customer and plan records, a LinkedList for usage history, a HashMap mapping SIM number to customer, and a TreeMap keeping bills ordered by date. Plan types are modelled with an enum (PREPAID, POSTPAID), and invoice text is built with StringBuilder. A custom ValidationException and a Validator class guard every input. The

application is preloaded with 20 customers, 6 plans and sample usage, and exposes eight modules through a black-and-gold JavaFX dashboard.

3. Case Study Design

The system follows a layered design so that user-interface code, business rules and data storage stay separate and easy to maintain.

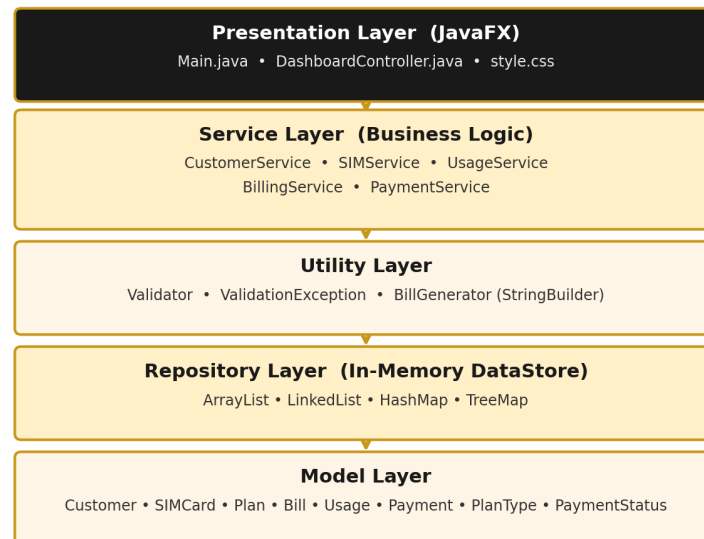


Figure 1: Layered architecture of Telecom BSS

3.1 Package Structure

Package / File	Responsibility
<code>com.telecom.Main</code>	JavaFX entry point: seeds data, loads the dashboard and the CSS theme (1280×800 window).
<code>model</code>	Customer, SIMCard, Plan, Bill, Usage, Payment, and the enums PlanType and PaymentStatus.
<code>repository.DataStore</code>	Static in-memory collections and sample-data seeding.
<code>service</code>	CustomerService, SIMService, UsageService, BillingService, PaymentService — business logic.
<code>util</code>	Validator, ValidationException, BillGenerator (StringBuilder invoice).
<code>controller.DashboardController</code>	Builds the sidebar navigation and all eight module screens.
<code>resources/css/style.css</code>	Black and gold theme for the JavaFX interface.

3.2 Class Design

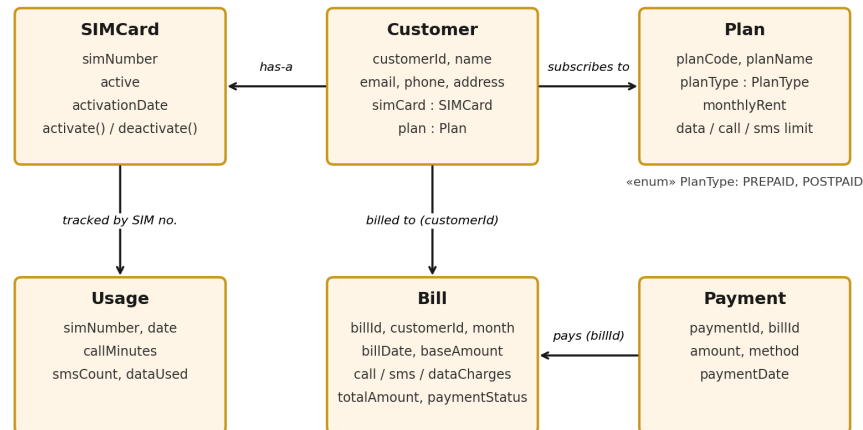


Figure 2: Relationships between the main model classes

Each Customer holds one SIMCard and one Plan. Usage records are linked to a SIM by its number, Bills are linked to a customer by customerId, and Payments reference a bill by billId.

3.3 Data Structures

Collection	Declaration in DataStore	Purpose
ArrayList	ArrayList<Customer>, ArrayList<Plan>, ArrayList<SIMCard>, ArrayList<Payment>	Stores customer, plan, SIM and payment records.
LinkedList	LinkedList<Usage>	Usage history; new records are inserted at the head using addFirst().
HashMap	HashMap<String, Customer>	SIM number → customer lookup in O(1) time.
TreeMap	TreeMap<LocalDate, Bill>	Bills kept sorted by bill date.

3.4 Application Modules

The sidebar exposes the seven modules expected by the case study, plus a dashboard overview:

#	Module	Functions
1	SIM Registration	Select customer and plan, enter a 10-digit SIM number, activate the SIM.
2	Plan Management	View plan table; create a plan with code, type, rent, data, call and SMS limits.
3	Usage Tracking	Add call minutes, SMS count and data (GB) for a date; view usage history.
4	Bill Generation	Generate a monthly bill and preview the StringBuilder-built invoice.
5	Payment Recording	Pay a pending bill by UPI, Card, Cash or Net Banking; view payment history.

#	Module	Functions
6	Customer Search	Add / edit / delete customers; live search by name, phone or SIM number.
7	Billing Reports	Bills sorted by total amount (highest first) with payment status.
–	Dashboard	Totals for customers, active SIMs, plans, generated bills and a customer table.

4. Methods, Algorithms & Technology Applied

4.1 Java Concepts Mapped to the Case Study

Java Concept	Application in the project	Where
Classes & Objects	Customer, SIMCard, Plan, Bill, Usage, Payment	model package
Constructors	Initialise objects, e.g. Plan(code, name, type, rent, data, calls, sms)	model package
Enum	PlanType (PREPAID, POSTPAID); PaymentStatus (PENDING, PAID)	PlanType.java, PaymentStatus.java
StringBuilder	Invoice text generation	BillGenerator.generate()
ArrayList	Customer, plan, SIM and payment records	DataStore
LinkedList	Usage history	DataStore, UsageService
HashMap	SIM number → Customer	DataStore, SIMService
TreeMap	Bills ordered by date	DataStore, BillingService
CRUD	Customer add/update/delete; plan creation	CustomerService, DashboardController
Searching	Search by name, phone or SIM	CustomerService.search()
Sorting	Customers by name; bills by total amount	CustomerService, showReports()
GUI	JavaFX dashboard (TableView, forms, dialogs)	DashboardController
Exception Handling	Custom checked ValidationException, NumberFormatException handling in UI	util, controller
Validation	SIM number, phone, email, plan code, usage, payment selection	Validator, PaymentService

Note on the GUI: the case study lists Swing as the GUI technology. This project uses JavaFX, the modern successor to Swing for Java desktop applications, to deliver the same telecom billing GUI with CSS styling and TableView components.

4.2 Key Algorithms

Bill calculation. The bill equals the plan's monthly rent plus charges for usage that exceeds the plan allowance. Usage is summed from the LinkedList for the customer's SIM number:

```
base          = plan.monthlyRent
callCharges   = max(0, totalCallMinutes - plan.callLimit) x Rs 0.50
smsCharges    = max(0, totalSms        - plan.smsLimit)   x Rs 0.20
dataCharges   = max(0, totalDataGB     - plan.dataLimit)  x Rs 10.00
total         = base + callCharges + smsCharges + dataCharges
```

Searching. CustomerService.search() converts the query to lower case and filters the ArrayList using a Java Stream, matching on name, phone number or SIM number, then sorts the matches alphabetically with a Comparator. Finding a customer directly from a SIM number uses the HashMap.

Sorting. Search results are sorted by customer name; the billing report sorts bills by total amount in descending order using
Comparator.comparingDouble(Bill::getTotalAmount).reversed().

Validation. Each input is checked against a rule before any record is created. Failures throw ValidationException, which the UI catches and shows in an alert dialog.

Input	Rule	Error message
Customer name	Not null or blank	Customer name is required.
Phone number	Exactly 10 digits	Phone number must contain exactly 10 digits.
Email	Regex pattern user@domain	Enter a valid email address.
SIM number	Exactly 10 digits and not already registered	SIM number must contain exactly 10 digits.
Plan code	2–12 letters, digits or hyphens	Plan code must be 2-12 letters, numbers or hyphens.
Usage values	Calls, SMS and data not negative; SIM must be registered	Usage values cannot be negative.
Payment	Bill selected, method selected, bill not already PAID	This bill is already paid.

4.3 Technology Stack

Category	Details
Language	Java 21
GUI framework	JavaFX 21.0.6 (javafx-controls, javafx-fxml)
Build tool	Apache Maven (javafx-maven-plugin 0.0.8), run with mvn clean javafx:run
Data storage	In-memory Java collections (no external database)
IDE	IntelliJ IDEA / Eclipse / VS Code
Version control	Git and GitHub: github.com/Yashhh710/telecom-bss-javafx

5. Implementation Details and Snapshots

5.1 Model Layer — Enum and Constructors

Plans are modelled with the PlanType enum, and every model class initialises its state through a constructor. A new SIMCard starts inactive and is activated with activate(), which records the activation date.

```
public enum PlanType { PREPAID, POSTPAID }

public class SIMCard {
    public SIMCard(String simNumber) {
        this.simNumber = simNumber;
        this.active = false;
    }
    public void activate() {
        active = true;
        activationDate = LocalDate.now();
    }
}
```

5.2 Repository Layer — Collections

```
public static final ArrayList<Customer> customers = new ArrayList<>();
public static final LinkedList<Usage> usageHistory = new LinkedList<>();
public static final HashMap<String, Customer> simCustomerMap = new HashMap<>();
public static final TreeMap<LocalDate, Bill> bills = new TreeMap<>();
```

DataSource.seed() loads six plans (P149 to P799, four prepaid and two postpaid), 20 customers from Navi Mumbai localities, 20 activated SIMs and 22 usage records for demonstration.

Code	Plan	Type	Rent (Rs)	Data (GB)	Calls / SMS
P149	Starter 149	PREPAID	149	28	2000 / 100
P199	Basic 199	PREPAID	199	45	3000 / 100
P299	Smart 299	PREPAID	299	60	5000 / 100
P399	Power 399	PREPAID	399	70	7500 / 150
P499	Max 499	POSTPAID	499	50	10000 / 300
P799	Premium 799	POSTPAID	799	90	20000 / 500

5.3 Service Layer — Validation and Exception Handling

Services validate input before changing data. SIMService.activate() is a representative example; it rejects malformed or duplicate SIM numbers and a missing plan.

```
public SIMCard activate(Customer c, String number, Plan plan) throws ValidationException {
    Validator.validateSIM(number);
    if (plan == null) throw new ValidationException("Select a plan.");
    if (DataSource.simCustomerMap.containsKey(number))
        throw new ValidationException("This SIM number is already registered.");
    SIMCard sim = new SIMCard(number);
    sim.activate();
    DataSource.sims.add(sim);
    DataSource.simCustomerMap.put(number, c);
    c.setSimCard(sim); c.setPlan(plan);
    return sim;
}
```

```
}
```

5.4 Bill Generation with StringBuilder

BillGenerator builds the invoice text one line at a time using StringBuilder and String.format for aligned currency values.

```
StringBuilder sb = new StringBuilder();
sb.append("TELECOM INVOICE\n");
sb.append("Bill ID      : ").append(bill.getBillId()).append("\n");
sb.append(String.format("Base Amount    : Rs%.2f\n", bill.getBaseAmount()));
sb.append(String.format("TOTAL        : Rs%.2f\n", bill.getTotalAmount()));
return sb.toString();
```

5.5 Application Snapshots

The snapshots below show each module of the application. They were captured from the of the project, which reproduces the JavaFX interface and workflow.

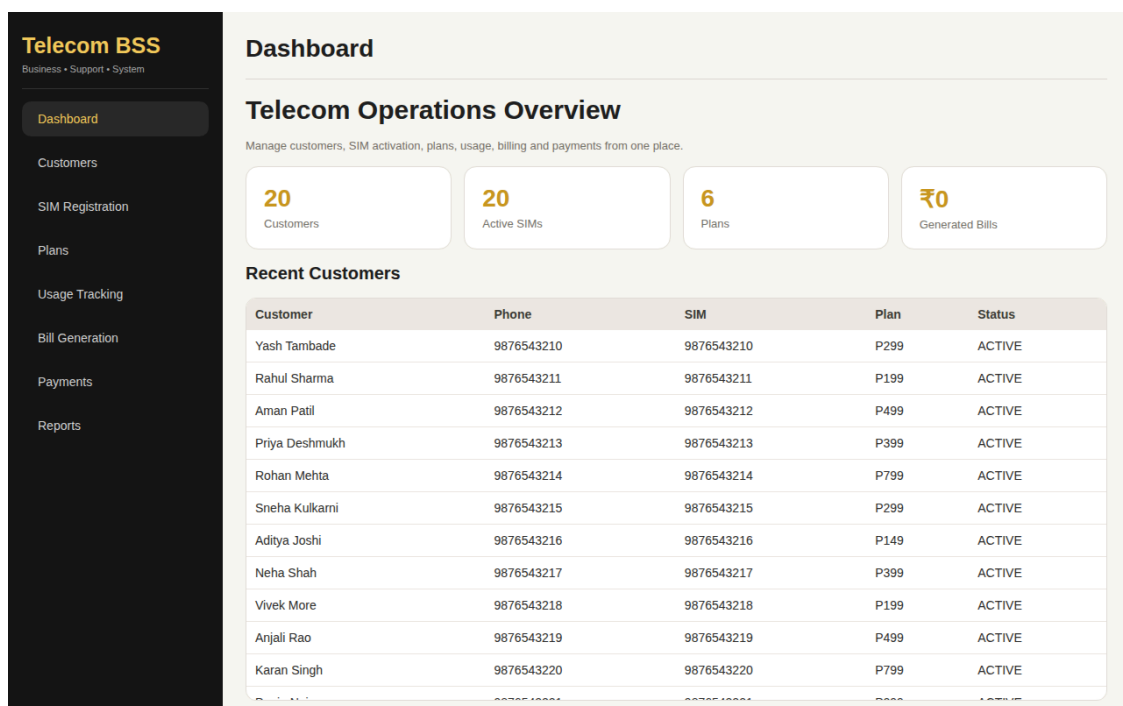


Figure 3: Dashboard — operations overview and customer table

Telecom BSS

Business • Support • System

Dashboard

Customers

SIM Registration

Plans

Usage Tracking

Bill Generation

Payments

Reports

Customer Management

Search by name, phone or SIM...

+ Add Customer

Edit

Delete

Customer	Phone	SIM	Plan	Status
Yash Tambade	9876543210	9876543210	P299	ACTIVE
Rahul Sharma	9876543211	9876543211	P199	ACTIVE
Aman Patil	9876543212	9876543212	P499	ACTIVE
Priya Deshmukh	9876543213	9876543213	P399	ACTIVE
Rohan Mehta	9876543214	9876543214	P799	ACTIVE
Sneha Kulkarni	9876543215	9876543215	P299	ACTIVE
Aditya Joshi	9876543216	9876543216	P149	ACTIVE
Neha Shah	9876543217	9876543217	P399	ACTIVE
Vivek More	9876543218	9876543218	P199	ACTIVE
Anjali Rao	9876543219	9876543219	P499	ACTIVE
Karan Singh	9876543220	9876543220	P799	ACTIVE

Figure 4: Customer management with search, add, edit and delete

Telecom BSS

Business • Support • System

Dashboard

Customers

SIM Registration

Plans

Usage Tracking

Bill Generation

Payments

Reports

SIM Registration & Activation

New SIM Activation

Customer

Yash Tambade (9876543210)

SIM Number

10 digit SIM number

Plan

P149 - Starter 149 (₹149.00)

Activate SIM

Validation: SIM number must contain exactly 10 digits and cannot already be registered.

Figure 5: SIM registration and activation

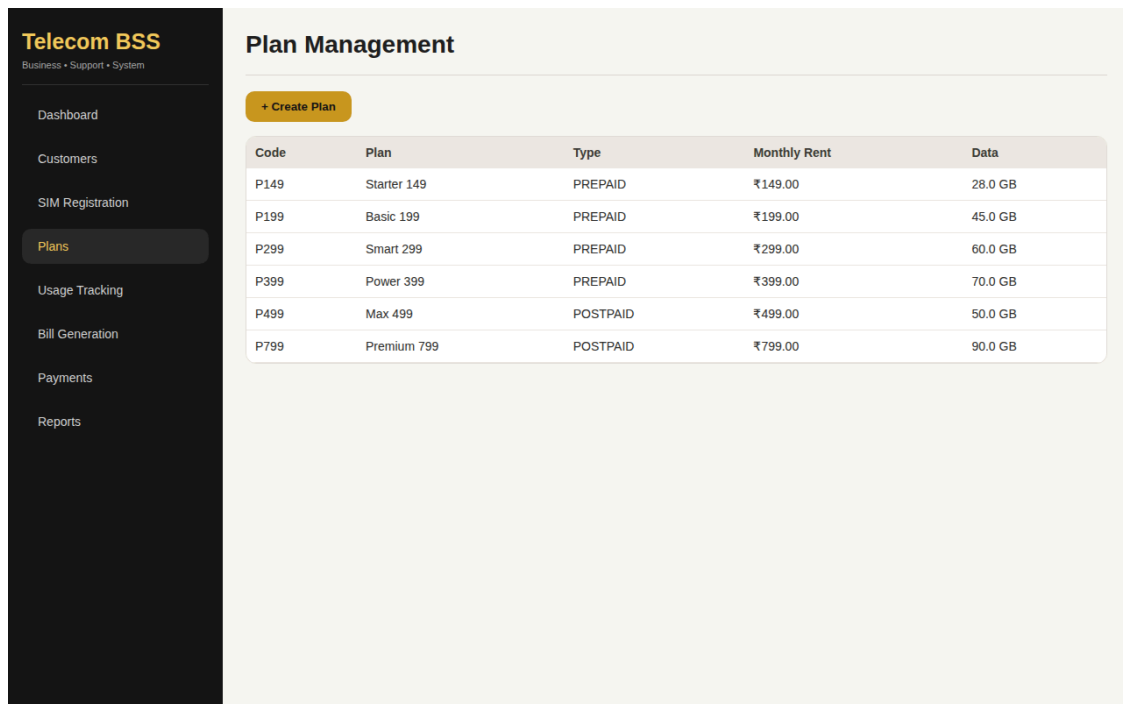


Figure 6: Plan management (prepaid and postpaid)

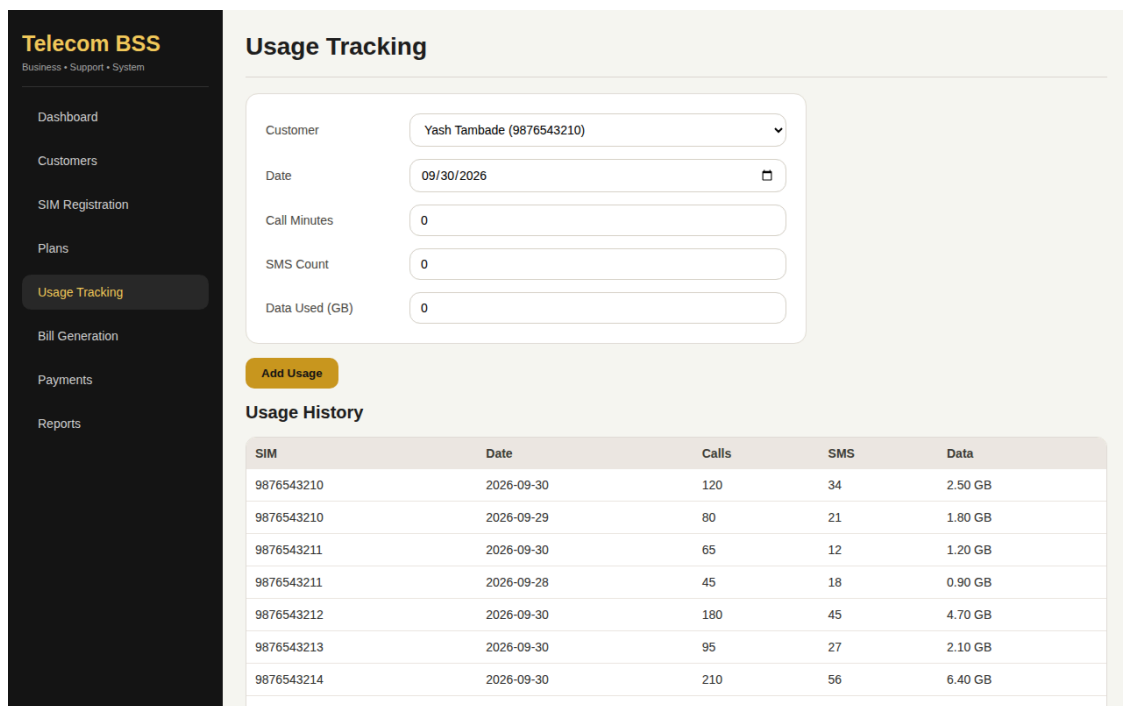


Figure 7: Usage tracking — calls, SMS and data

Telecom BSS

Business • Support • System

Dashboard

Customers

SIM Registration

Plans

Usage Tracking

Bill Generation

Payments

Reports

Bill Generation

Customer

Aman Patil (9876543212)

Billing Month

SEPTEMBER 2026

Generate Bill

Invoice Preview

```

=====
TELECOM INVOICE
=====
Bill ID       : BILL-1790803200455
Customer      : Aman Patil
Phone         : 9876543212
SIM           : 9876543212
Plan          : Max 499
Billing Month : SEPTEMBER 2026
-----
Base Amount   : ₹499.00
Call Charges  : ₹0.00
SMS Charges   : ₹0.00
Data Charges  : ₹0.00
-----
TOTAL         : ₹499.00
Status        : PENDING
=====

```

Figure 8: Bill generation with StringBuilder invoice preview

Telecom BSS

Business • Support • System

Dashboard

Customers

SIM Registration

Plans

Usage Tracking

Bill Generation

Payments

Reports

Payment Recording

Bill

BILL-1790803200455 • ₹499.00 • PENDING

Payment Method

UPI

Record Payment

Payment History

Transaction	Bill	Amount	Method
TXN-1790803202790	BILL-1790803201071	₹799.00	CARD

Figure 9: Payment recording

Telecom BSS Business • Support • System Dashboard Customers SIM Registration Plans Usage Tracking Bill Generation Payments Reports	Billing Reports			
	Bills are sorted by amount in descending order using Java sorting and the TreeMap date index.			
	Bill ID	Date	Amount	Status
	BILL-1790803201071	2026-09-30	₹799.00	PAID
	BILL-1790803202137	2026-09-30	₹799.00	PENDING
	BILL-1790803200455	2026-09-30	₹499.00	PENDING
	BILL-1790803201604	2026-09-30	₹299.00	PENDING

Figure 10: Billing reports

6. Results and Conclusion

Findings

- All seven expected modules — SIM registration, plan management, usage tracking, bill generation, payment recording, customer search and billing reports — are implemented and reachable from a single dashboard.
- Every Java concept required by the case study is used in a meaningful place: enums for plan types and payment status, StringBuilder for invoices, and the four collections ArrayList, LinkedList, HashMap and TreeMap for their respective roles.
- Input validation and a custom checked exception prevent invalid phone numbers, SIM numbers, plan codes, negative usage and double payment of a bill.
- Layered design keeps the GUI, business logic and storage separate, which makes each part easy to test and extend.

Limitations

- Data is stored in memory only and resets to the sample data when the application restarts.
- Bills are keyed by date in the TreeMap, so two bills generated on the same day would overwrite each other; a composite key (date + bill ID) would fix this.
- Prepaid and postpaid plans currently share the same overage formula; plan type is stored and displayed but does not yet change the billing rules.
- Usage is summed across all recorded dates rather than filtered by billing month.

Future Scope

- Database persistence (MySQL/PostgreSQL), user login with admin and employee roles.
- Separate prepaid (balance deduction) and postpaid (monthly invoice) billing rules.
- PDF invoices, e-mail/SMS bill delivery and revenue charts.

Conclusion

The Telecom BSS project delivers a complete, working telecom SIM activation and billing workflow in Java and JavaFX. It applies object-oriented design, the Java Collections Framework, enum-based plan modelling, StringBuilder-based bill formatting, validation and exception handling to a realistic problem, and provides a solid foundation for a production-grade system with persistent storage and richer billing rules.

7. References

- Oracle, Java SE 21 Documentation — java.util Collections Framework (ArrayList, LinkedList, HashMap, TreeMap), docs.oracle.com.
- OpenJFX, JavaFX 21 Documentation and API, openjfx.io.
- Apache Maven Project, Maven Getting Started Guide, maven.apache.org.
- Oracle, The Java Tutorials — Enum Types, Exceptions, and Strings/StringBuilder.
- Project source code and web demo: github.com/Yashhh710/telecom-bss-javafx.
- ITM Skills University, Java Programming Case Study 60: Telecom SIM Card Activation & Billing System (problem statement).

8. GitHub Repository

<https://github.com/Yashhh710/telecom-bss-javafx>